Distributional Semantics, Word Vectors & Neural Networks

From Co-occurrence to Embeddings

Timothy Obiso

NLP II Recitation
Brandeis University

February 6, 2026

"Nets are for the fish. Once you've gotten the fish, you can forget the net. Words are for meaning. Once you've gotten the meaning, you can forget the words."

荃者所以在鱼，得鱼而忘荃；言者所以在意，得意而忘言。

— *Zhuangzi, External Things (c. 3rd century BCE)*

Today's Goals

You've done the readings (Eisenstein Ch. 14, Mikolov et al. 2013, Pennington et al. 2014).

Today we'll:

- Review distributional semantics: co-occurrence, PMI, SVD
- Derive the Word2Vec skip-gram objective and negative sampling
- Understand GloVe and how it bridges count-based and prediction-based methods
- Connect word embeddings to neural networks
- See why all of these ideas are really the same thing

The Distributional Hypothesis

"You shall know a word by the company it keeps."

— J.R. Firth (1957)

The Distributional Hypothesis

"You shall know a word by the company it keeps."

— J.R. Firth (1957)

- Words that occur in similar **contexts** tend to have similar **meanings**
- We can represent word meaning as a vector over contexts
- No hand-crafted features — meaning emerges from **usage patterns**

The Distributional Hypothesis

"You shall know a word by the company it keeps."

— J.R. Firth (1957)

- Words that occur in similar **contexts** tend to have similar **meanings**
- We can represent word meaning as a vector over contexts
- No hand-crafted features — meaning emerges from **usage patterns**

Key insight (Eisenstein Ch. 14): Meaning is not a fixed dictionary definition — it's a **distribution** over the contexts in which a word appears.

Co-occurrence Matrices

Term–Context Matrix: Each row is a word, each column is a context.

Co-occurrence Matrices

Term–Context Matrix: Each row is a word, each column is a context.

	river	*money*	*water*	*loan*	*fish*	*deposit*
bank	3	8	2	7	1	9
stream	5	0	7	0	4	0
credit	0	6	0	5	0	4

Co-occurrence Matrices

Term–Context Matrix: Each row is a word, each column is a context.

	river	*money*	*water*	*loan*	*fish*	*deposit*
bank	3	8	2	7	1	9
stream	5	0	7	0	4	0
credit	0	6	0	5	0	4

- **Context** can be: entire documents, fixed-size windows, or syntactic dependencies
- Raw counts are dominated by frequent words — we need **weighting**
- "stream" looks like river-bank, "credit" looks like money-bank

PMI and PPMI

Pointwise Mutual Information (PMI):

$$\text{PMI}(w, c) = \log \frac{P(w, c)}{P(w) \cdot P(c)}$$

PMI and PPMI

Pointwise Mutual Information (PMI):

$$\text{PMI}(w, c) = \log \frac{P(w, c)}{P(w) \cdot P(c)}$$

- Measures how much more (or less) w and c co-occur than expected by chance
- PMI > 0: words co-occur more than expected
- PMI < 0: words co-occur less than expected
- PMI $= 0$: independence

PMI and PPMI

Pointwise Mutual Information (PMI):

$$\text{PMI}(w, c) = \log \frac{P(w, c)}{P(w) \cdot P(c)}$$

- Measures how much more (or less) w and c co-occur than expected by chance
- PMI > 0: words co-occur more than expected
- PMI < 0: words co-occur less than expected
- PMI $= 0$: independence

Positive PMI (PPMI):

$$\text{PPMI}(w, c) = \max\big(0,\ \text{PMI}(w, c)\big)$$

Zeroes out negative values (which are unreliable with sparse data).

Dimensionality Reduction with SVD

The PPMI matrix is **huge and sparse**. We want dense, low-dimensional vectors.

Dimensionality Reduction with SVD

The PPMI matrix is **huge and sparse**. We want dense, low-dimensional vectors.

Truncated SVD:

$$\mathbf{M} \approx \mathbf{U}_k \, \mathbf{\Sigma}_k \, \mathbf{V}_k^\top$$

- $\mathbf{U}_k \in \mathbb{R}^{|V| \times k}$ — word vectors (keep these!)
- $\mathbf{\Sigma}_k$ — diagonal matrix of top-k singular values
- $\mathbf{V}_k$ — context vectors
- Typical k: 100–300 dimensions

Dimensionality Reduction with SVD

The PPMI matrix is **huge and sparse**. We want dense, low-dimensional vectors.

Truncated SVD:

$$\mathbf{M} \approx \mathbf{U}_k \, \mathbf{\Sigma}_k \, \mathbf{V}_k^\top$$

- $\mathbf{U}_k \in \mathbb{R}^{|V| \times k}$ — word vectors (keep these!)
- $\mathbf{\Sigma}_k$ — diagonal matrix of top-k singular values
- $\mathbf{V}_k$ — context vectors
- Typical k: 100–300 dimensions

Why SVD? Captures latent semantic structure. Words that never co-occur directly can still end up with similar vectors if they share second-order co-occurrence patterns.

Practice: Computing PMI

Exercise: Given counts from a corpus of $N = 1000$ word–context pairs:

	coffee	*data*	Row Total
drink	15	1	50
analyze	2	20	60
Col Total	40	80	

Compute:

1. PMI(drink, coffee)
2. PMI(analyze, data)
3. PPMI(drink, data)

Hint: $P(w, c) = \frac{\text{count}(w,c)}{N}$, $P(w) = \frac{\text{row total}}{N}$, $P(c) = \frac{\text{col total}}{N}$

Practice: PMI Solutions

1. PMI(drink, coffee):

$$\log \frac{15/1000}{(50/1000)(40/1000)} = \log \frac{0.015}{0.002} = \log 7.5 \approx 2.01$$

Practice: PMI Solutions

1. PMI(drink, coffee):

$$\log \frac{15/1000}{(50/1000)(40/1000)} = \log \frac{0.015}{0.002} = \log 7.5 \approx 2.01$$

2. PMI(analyze, data):

$$\log \frac{20/1000}{(60/1000)(80/1000)} = \log \frac{0.02}{0.0048} = \log 4.17 \approx 1.43$$

Practice: PMI Solutions

1. PMI(drink, coffee):

$$\log \frac{15/1000}{(50/1000)(40/1000)} = \log \frac{0.015}{0.002} = \log 7.5 \approx 2.01$$

2. PMI(analyze, data):

$$\log \frac{20/1000}{(60/1000)(80/1000)} = \log \frac{0.02}{0.0048} = \log 4.17 \approx 1.43$$

3. PMI(drink, data):

$$\log \frac{1/1000}{(50/1000)(80/1000)} = \log \frac{0.001}{0.004} = \log 0.25 \approx -1.39$$

$\text{PPMI} = \max(0, -1.39) = \mathbf{0}$

Practice: PMI Edge Cases

Harder problems:

1. Suppose the word "the" appears in *every* context in your corpus. What is PMI(the, c) for any context c? Why is this a problem?

Practice: PMI Edge Cases

Harder problems:

1. Suppose the word "the" appears in *every* context in your corpus. What is PMI(the, c) for any context c? Why is this a problem?

2. You observe count(quizzify, exam) $= 1$ in a corpus of $N = 10{,}000{,}000$ pairs, with count(quizzify) $= 1$ and count(exam) $= 500$.
Compute PMI(quizzify, exam). Is this value meaningful? What does this tell you about PMI and rare words?

Practice: PMI Edge Cases

Harder problems:

1. Suppose the word "the" appears in *every* context in your corpus. What is $\text{PMI}(\text{the}, c)$ for any context c? Why is this a problem?

2. You observe $\text{count}(\text{quizzify}, \text{exam}) = 1$ in a corpus of $N = 10{,}000{,}000$ pairs, with $\text{count}(\text{quizzify}) = 1$ and $\text{count}(\text{exam}) = 500$.
Compute $\text{PMI}(\text{quizzify}, \text{exam})$. Is this value meaningful? What does this tell you about PMI and rare words?

3. You double your corpus by duplicating it. What happens to:

1. The raw co-occurrence counts?
2. The PMI values?
3. The PPMI matrix?

Hint: Think about what happens to $P(w, c)$, $P(w)$, and $P(c)$.

Practice: PMI Edge Cases — Solutions

1. If "the" appears in every context: $P(\text{the}) \approx 1$, so $\text{PMI}(\text{the}, c) = \log \frac{P(\text{the},c)}{P(\text{the}) \cdot P(c)} \approx \log \frac{P(c)}{1 \cdot P(c)} = \log 1 = 0$ for all c. The vector is all zeros — "the" looks equally related to everything, which is useless.

Practice: PMI Edge Cases — Solutions

1. If "the" appears in every context: $P(\text{the}) \approx 1$, so
$\text{PMI}(\text{the}, c) = \log \frac{P(\text{the},c)}{P(\text{the}) \cdot P(c)} \approx \log \frac{P(c)}{1 \cdot P(c)} = \log 1 = 0$ for all c. The vector is all zeros — "the" looks equally related to everything, which is useless.

2. $\text{PMI} = \log \frac{1/10^7}{(1/10^7)(500/10^7)} = \log \frac{10^7}{500} = \log 20000 \approx 9.9$
Extremely high PMI, but based on a single observation! PMI has a **bias toward rare words** — low-frequency events that co-occur even once get inflated scores. This is why people use smoothing or frequency thresholds.

Practice: PMI Edge Cases — Solutions

1. If "the" appears in every context: $P(\text{the}) \approx 1$, so $\text{PMI}(\text{the}, c) = \log \frac{P(\text{the},c)}{P(\text{the}) \cdot P(c)} \approx \log \frac{P(c)}{1 \cdot P(c)} = \log 1 = 0$ for all c. The vector is all zeros — "the" looks equally related to everything, which is useless.

2. $\text{PMI} = \log \frac{1/10^7}{(1/10^7)(500/10^7)} = \log \frac{10^7}{500} = \log 20000 \approx 9.9$
Extremely high PMI, but based on a single observation! PMI has a **bias toward rare words** — low-frequency events that co-occur even once get inflated scores. This is why people use smoothing or frequency thresholds.

3. All counts double, but all probabilities stay the same (they're ratios). PMI and PPMI are **completely unchanged**. This is a nice property — PMI is invariant to corpus size (though variance of estimates decreases).

Discussion: Limitations of Count-Based Methods

Discuss with your neighbor (2 min): What are the practical and theoretical limitations of using PPMI + SVD for word representations?

Discussion: Limitations of Count-Based Methods

Discuss with your neighbor (2 min): What are the practical and theoretical limitations of using PPMI + SVD for word representations?

Some things to consider:

- What happens with rare words?
- How does the choice of context window affect the vectors?
- How expensive is SVD on a large vocabulary?
- Can these vectors capture word order or compositionality?
- What do negative PMI values tell us, and why do we discard them?

Word2Vec: From Counts to Prediction

Key idea: Instead of counting co-occurrences, **predict** context from words (or vice versa).

Word2Vec: From Counts to Prediction

Key idea: Instead of counting co-occurrences, **predict** context from words (or vice versa).

Two architectures (Mikolov et al., 2013):

Skip-gram:

Given center word w_t,
predict context words w_{t+j}

$$\max \sum_{t=1}^{T} \sum_{\substack{j=-c \\ j \neq 0}}^{c} \log P(w_{t+j} | w_t)$$

CBOW:

Given context words,
predict center word w_t

$$\max \sum_{t=1}^{T} \log P(w_t | w_{t-c}, \ldots, w_{t+c})$$

Word2Vec: From Counts to Prediction

Key idea: Instead of counting co-occurrences, **predict** context from words (or vice versa).

Two architectures (Mikolov et al., 2013):

Skip-gram:

Given center word w_t,
predict context words w_{t+j}

$$\max \sum_{t=1}^{T} \sum_{\substack{j=-c \\ j \neq 0}}^{c} \log P(w_{t+j} | w_t)$$

CBOW:

Given context words,
predict center word w_t

$$\max \sum_{t=1}^{T} \log P(w_t | w_{t-c}, \ldots, w_{t+c})$$

- Window size c is a hyperparameter (typically 5–10)
- Skip-gram works better for rare words; CBOW is faster

Skip-gram: Softmax Objective

For a center word w and context word c:

$$P(c|w) = \frac{\exp(\mathbf{u}_c^\top \mathbf{v}_w)}{\sum_{c' \in V} \exp(\mathbf{u}_{c'}^\top \mathbf{v}_w)}$$

Skip-gram: Softmax Objective

For a center word w and context word c:

$$P(c|w) = \frac{\exp(\mathbf{u}_c^\top \mathbf{v}_w)}{\sum_{c' \in V} \exp(\mathbf{u}_{c'}^\top \mathbf{v}_w)}$$

- $\mathbf{v}_w$ — **center** word embedding (input matrix $\mathbf{W}$)
- $\mathbf{u}_c$ — **context** word embedding (output matrix $\mathbf{W}'$)
- Two separate embedding matrices!

Skip-gram: Softmax Objective

For a center word w and context word c:

$$P(c|w) = \frac{\exp(\mathbf{u}_c^\top \mathbf{v}_w)}{\sum_{c' \in V} \exp(\mathbf{u}_{c'}^\top \mathbf{v}_w)}$$

- $\mathbf{v}_w$ — **center** word embedding (input matrix $\mathbf{W}$)
- $\mathbf{u}_c$ — **context** word embedding (output matrix $\mathbf{W}'$)
- Two separate embedding matrices!

Problem: The denominator sums over **entire vocabulary** $|V|$ (often 100K+ words). Way too expensive to compute at every training step.

Negative Sampling

Solution: Replace the full softmax with a binary classification task.

Negative Sampling

Solution: Replace the full softmax with a binary classification task.

Objective (for one positive pair (w, c) and k negative samples):

$$\mathcal{L} = \log \sigma(\mathbf{u}_c^\top \mathbf{v}_w) + \sum_{i=1}^{k} \log \sigma(-\mathbf{u}_{c_i}^\top \mathbf{v}_w), \quad c_i \sim P_n$$

Negative Sampling

Solution: Replace the full softmax with a binary classification task.

Objective (for one positive pair (w, c) and k negative samples):

$$\mathcal{L} = \log \sigma(\mathbf{u}_c^\top \mathbf{v}_w) + \sum_{i=1}^{k} \log \sigma(-\mathbf{u}_{c_i}^\top \mathbf{v}_w), \quad c_i \sim P_n$$

- $\sigma(x) = \frac{1}{1+e^{-x}}$ (sigmoid — you know this one!)
- Push **real** context words closer to center word
- Push **random** (negative) words away from center word
- $P_n(w) \propto f(w)^{3/4}$ — smoothed unigram distribution
- Typical k: 5–20 negative samples

What Word2Vec Captures

The learned vectors exhibit remarkable **linear structure**:

What Word2Vec Captures

The learned vectors exhibit remarkable **linear structure**:

Word Analogies:

$$\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}} \approx \mathbf{v}_{\text{queen}}$$

What Word2Vec Captures

The learned vectors exhibit remarkable **linear structure**:

Word Analogies:

$$\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}} \approx \mathbf{v}_{\text{queen}}$$

More examples:

- $\text{Paris} - \text{France} + \text{Italy} \approx \text{Rome}$ (geography)
- $\text{walking} - \text{walk} + \text{swim} \approx \text{swimming}$ (morphology)
- $\text{bigger} - \text{big} + \text{small} \approx \text{smaller}$ (comparative)

What Word2Vec Captures

The learned vectors exhibit remarkable **linear structure**:

Word Analogies:

$$\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}} \approx \mathbf{v}_{\text{queen}}$$

More examples:

- $\text{Paris} - \text{France} + \text{Italy} \approx \text{Rome}$ (geography)
- $\text{walking} - \text{walk} + \text{swim} \approx \text{swimming}$ (morphology)
- $\text{bigger} - \text{big} + \text{small} \approx \text{smaller}$ (comparative)

Cosine similarity measures semantic relatedness:

$$\text{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \, \|\mathbf{v}\|}$$

Practice: Word Analogies & Cosine Similarity

Exercise 1: Analogies

Using the analogy formula $\mathbf{a} - \mathbf{b} + \mathbf{c} \approx \mathbf{d}$, what would you expect for:

1. $\text{Tokyo} - \text{Japan} + \text{Germany} \approx ?$
2. $\text{went} - \text{go} + \text{eat} \approx ?$
3. $\text{queen} - \text{woman} + \text{man} \approx ?$

Exercise 2: Cosine Similarity

Given 2D vectors: $\mathbf{v}_{\text{cat}} = (3, 4), \quad \mathbf{v}_{\text{dog}} = (4, 3), \quad \mathbf{v}_{\text{car}} = (-2, 5).$

1. Compute $\text{sim}(\text{cat}, \text{dog})$
2. Compute $\text{sim}(\text{cat}, \text{car})$
3. Which pair is more semantically similar?

Practice: Cosine Similarity Solutions

Analogies: 1. Berlin 2. ate 3. king

Practice: Cosine Similarity Solutions

Analogies: 1. Berlin 2. ate 3. king

Cosine Similarity:

$$\text{sim}(\text{cat}, \text{dog}) = \frac{(3)(4)+(4)(3)}{\sqrt{9+16}\cdot\sqrt{16+9}} = \frac{24}{25} = \mathbf{0.96}$$

Practice: Cosine Similarity Solutions

Analogies: 1. Berlin 2. ate 3. king

Cosine Similarity:

$$\text{sim}(\text{cat}, \text{dog}) = \frac{(3)(4)+(4)(3)}{\sqrt{9+16}\cdot\sqrt{16+9}} = \frac{24}{25} = \mathbf{0.96}$$

$$\text{sim}(\text{cat}, \text{car}) = \frac{(3)(-2)+(4)(5)}{\sqrt{25}\cdot\sqrt{4+25}} = \frac{14}{5\cdot\sqrt{29}} \approx \frac{14}{26.93} \approx \mathbf{0.52}$$

Practice: Cosine Similarity Solutions

Analogies: 1. Berlin 2. ate 3. king

Cosine Similarity:

$$\text{sim}(\text{cat}, \text{dog}) = \frac{(3)(4)+(4)(3)}{\sqrt{9+16}\cdot\sqrt{16+9}} = \frac{24}{25} = \mathbf{0.96}$$

$$\text{sim}(\text{cat}, \text{car}) = \frac{(3)(-2)+(4)(5)}{\sqrt{25}\cdot\sqrt{4+25}} = \frac{14}{5\cdot\sqrt{29}} \approx \frac{14}{26.93} \approx \mathbf{0.52}$$

- cat–dog similarity (0.96) $\gg$ cat–car similarity (0.52) ✓
- Cosine similarity captures semantic relatedness even in 2D

Activity: Semantle

Semantle is a word-guessing game built on Word2Vec cosine similarity.

Rules:

- There's a secret target word
- You guess words, and the game tells you the **cosine similarity** to the target
- Closer to 1.0 = closer in embedding space = "warmer"

Activity: Semantle

Semantle is a word-guessing game built on Word2Vec cosine similarity.

Rules:

- There's a secret target word
- You guess words, and the game tells you the **cosine similarity** to the target
- Closer to 1.0 = closer in embedding space = "warmer"

Let's play a round together: `https://semantle.com`

Activity: Semantle

Semantle is a word-guessing game built on Word2Vec cosine similarity.

Rules:

- There's a secret target word
- You guess words, and the game tells you the **cosine similarity** to the target
- Closer to 1.0 = closer in embedding space = "warmer"

Let's play a round together: `https://semantle.com`

While we play, think about:

- What strategy are you using? Why does it work (or not)?
- When does cosine similarity match your intuition? When doesn't it?
- What kinds of relationships does Word2Vec capture vs. miss?
- Why might a synonym score lower than a related-but-different word?

Semantle Debrief

Discuss (2 min): What did the game reveal about word embeddings?

Semantle Debrief

Discuss (2 min): What did the game reveal about word embeddings?

Common observations:

- **Topical similarity $\neq$ synonymy:** "hospital" and "doctor" can be closer than "doctor" and "physician"
- **Frequency matters:** Common words tend to cluster differently than rare ones
- **Polysemy hurts:** "bank" is close to both "river" and "money" words
- **Morphology helps:** Different forms of the same word (run, running, ran) are often nearby

Semantle Debrief

Discuss (2 min): What did the game reveal about word embeddings?

Common observations:

- **Topical similarity $\neq$ synonymy:** "hospital" and "doctor" can be closer than "doctor" and "physician"
- **Frequency matters:** Common words tend to cluster differently than rare ones
- **Polysemy hurts:** "bank" is close to both "river" and "money" words
- **Morphology helps:** Different forms of the same word (run, running, ran) are often nearby

This is exactly why understanding *what* these vectors encode matters — and why we moved beyond static embeddings.

Practice: Word2Vec Tricky Cases

1. **Cosine vs. Euclidean:** Given vectors $\mathbf{v}_a = (1, 0)$, $\mathbf{v}_b = (100, 0)$, $\mathbf{v}_c = (0.6, 0.8)$.
   - Compute cosine similarity for all three pairs
   - Compute Euclidean distance for all three pairs
   - The two metrics disagree about which words are similar. Why? Which is more appropriate for word embeddings?

Practice: Word2Vec Tricky Cases

1. **Cosine vs. Euclidean:** Given vectors $\mathbf{v}_a = (1, 0)$, $\mathbf{v}_b = (100, 0)$, $\mathbf{v}_c = (0.6, 0.8)$.
   - Compute cosine similarity for all three pairs
   - Compute Euclidean distance for all three pairs
   - The two metrics disagree about which words are similar. Why? Which is more appropriate for word embeddings?

2. **Window size matters:** Predict how Word2Vec embeddings would differ between:
   - Window size $c = 2$ (small, local context)
   - Window size $c = 10$ (large, broad context)

Which would give higher similarity between "dog" and "cat"? Between "dog" and "walked"? Why?

Practice: Word2Vec Tricky Cases

1. Cosine vs. Euclidean: Given vectors $\mathbf{v}_a = (1, 0)$, $\mathbf{v}_b = (100, 0)$, $\mathbf{v}_c = (0.6, 0.8)$.

- Compute cosine similarity for all three pairs
- Compute Euclidean distance for all three pairs
- The two metrics disagree about which words are similar. Why? Which is more appropriate for word embeddings?

2. Window size matters: Predict how Word2Vec embeddings would differ between:

- Window size $c = 2$ (small, local context)
- Window size $c = 10$ (large, broad context)

Which would give higher similarity between "dog" and "cat"? Between "dog" and "walked"? Why?

3. You train Word2Vec on a corpus where "hot dog" appears 10,000 times. How does this affect the vector for "hot"? For "dog"? Is this a problem?

Practice: Word2Vec Tricky Cases — Solutions

1. $\text{sim}(a, b) = 1.0$ (same direction), $\text{sim}(a, c) = 0.6$, $\text{sim}(b, c) = 0.6$
$\text{dist}(a, b) = 99$ (huge!), $\text{dist}(a, c) = 0.89$, $\text{dist}(b, c) = 99.2$
Euclidean says a and c are closest; cosine says a and b are identical. Cosine is better because it measures **direction** (semantic similarity), not **magnitude** (which just reflects word frequency — frequent words get larger gradient updates).

Practice: Word2Vec Tricky Cases — Solutions

1. $\text{sim}(a, b) = 1.0$ (same direction), $\text{sim}(a, c) = 0.6$, $\text{sim}(b, c) = 0.6$
$\text{dist}(a, b) = 99$ (huge!), $\text{dist}(a, c) = 0.89$, $\text{dist}(b, c) = 99.2$
Euclidean says a and c are closest; cosine says a and b are identical. Cosine is better because it measures **direction** (semantic similarity), not **magnitude** (which just reflects word frequency — frequent words get larger gradient updates).

2. Small windows capture **syntactic** similarity (words that can substitute for each other: "dog" $\approx$ "cat"). Large windows capture **topical** similarity (words from the same domain: "dog" $\approx$ "leash", "bark"). So $c = 2$ gives higher dog–cat similarity; $c = 10$ gives higher dog–walked similarity.

Practice: Word2Vec Tricky Cases — Solutions

1. $\text{sim}(a, b) = 1.0$ (same direction), $\text{sim}(a, c) = 0.6$, $\text{sim}(b, c) = 0.6$
$\text{dist}(a, b) = 99$ (huge!), $\text{dist}(a, c) = 0.89$, $\text{dist}(b, c) = 99.2$
Euclidean says a and c are closest; cosine says a and b are identical. Cosine is better because it measures **direction** (semantic similarity), not **magnitude** (which just reflects word frequency — frequent words get larger gradient updates).

2. Small windows capture **syntactic** similarity (words that can substitute for each other: "dog" $\approx$ "cat"). Large windows capture **topical** similarity (words from the same domain: "dog" $\approx$ "leash", "bark"). So $c = 2$ gives higher dog–cat similarity; $c = 10$ gives higher dog–walked similarity.

3. "hot" gets pulled toward food contexts; "dog" gets pulled toward food/condiment contexts. Both vectors become **contaminated** by the multi-word expression. This is a general problem with token-level embeddings — they can't distinguish compositional from non-compositional phrases.

Discussion: Negative Sampling

Discuss with your neighbor (2 min): Why does negative sampling work as a training objective? What tradeoffs does it introduce?

Discussion: Negative Sampling

Discuss with your neighbor (2 min): Why does negative sampling work as a training objective? What tradeoffs does it introduce?

Consider:

- How does it approximate the full softmax?
- What happens if you sample too few negatives? Too many?
- Why use $P_n(w) \propto f(w)^{3/4}$ instead of uniform sampling?
- Could a "negative" sample actually be a valid context word?
- How does the number of negatives relate to implicit matrix factorization? (Levy & Goldberg, 2014: skip-gram with k negatives $\approx$ factorizing a shifted PMI matrix)

GloVe: Global Vectors for Word Representation

Pennington et al. (2014): Combine the best of count-based and prediction-based methods.

GloVe: Global Vectors for Word Representation

Pennington et al. (2014): Combine the best of count-based and prediction-based methods.

Core idea: Word vector differences should predict **co-occurrence ratios**.

	$P(\cdot\|\text{ice})$	$P(\cdot\|\text{steam})$
solid	high	low
gas	low	high
water	high	high
random	low	low

GloVe: Global Vectors for Word Representation

Pennington et al. (2014): Combine the best of count-based and prediction-based methods.

Core idea: Word vector differences should predict **co-occurrence ratios**.

	$P(\cdot\|\text{ice})$	$P(\cdot\|\text{steam})$
solid	high	low
gas	low	high
water	high	high
random	low	low

The **ratio** $\frac{P(\text{solid}|\text{ice})}{P(\text{solid}|\text{steam})}$ is large — this encodes meaning.

GloVe trains vectors so that: $\mathbf{w}_i^\top \mathbf{w}_j \approx \log X_{ij}$

GloVe Objective

Weighted least-squares on log co-occurrences:

$$J = \sum_{i,j=1}^{|V|} f(X_{ij}) \left(\mathbf{w}_i^\top \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2$$

GloVe Objective

Weighted least-squares on log co-occurrences:

$$J = \sum_{i,j=1}^{|V|} f(X_{ij}) \left(\mathbf{w}_i^\top \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2$$

- X_{ij} — co-occurrence count of words i and j
- $\mathbf{w}_i$, $\tilde{\mathbf{w}}_j$ — word and context vectors
- b_i, $\tilde{b}_j$ — bias terms
- $f(X_{ij})$ — weighting function that caps frequent pairs:

$$f(x) = \begin{cases} (x/x_{\max})^\alpha & \text{if } x < x_{\max} \\ 1 & \text{otherwise} \end{cases}$$

GloVe Objective

Weighted least-squares on log co-occurrences:

$$J = \sum_{i,j=1}^{|V|} f(X_{ij}) \left(\mathbf{w}_i^\top \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2$$

- X_{ij} — co-occurrence count of words i and j
- $\mathbf{w}_i$, $\tilde{\mathbf{w}}_j$ — word and context vectors
- b_i, $\tilde{b}_j$ — bias terms
- $f(X_{ij})$ — weighting function that caps frequent pairs:

$$f(x) = \begin{cases} (x/x_{\max})^\alpha & \text{if } x < x_{\max} \\ 1 & \text{otherwise} \end{cases}$$

Typically $x_{\max} = 100$, $\alpha = 0.75$. Prevents very frequent pairs from dominating.

GloVe vs Word2Vec vs SVD

	SVD on PPMI	**Word2Vec**	**GloVe**
Type	Count-based	Prediction-based	Hybrid
Input	Full co-occ. matrix	Local windows	Full co-occ. matrix
Training	Matrix factorization	SGD on windows	Weighted least squares
Scales	Poorly ($O(\|V\|^3)$)	Well (online)	Well (sparse matrix)
Global	Yes	No (local only)	Yes

GloVe vs Word2Vec vs SVD

	SVD on PPMI	**Word2Vec**	**GloVe**
Type	Count-based	Prediction-based	Hybrid
Input	Full co-occ. matrix	Local windows	Full co-occ. matrix
Training	Matrix factorization	SGD on windows	Weighted least squares
Scales	Poorly ($O(\|V\|^3)$)	Well (online)	Well (sparse matrix)
Global	Yes	No (local only)	Yes

Key Connection (Levy & Goldberg, 2014): Word2Vec with negative sampling is *implicitly* factorizing a shifted PMI matrix. These methods are more similar than they first appear!

Practice: GloVe Objective

Exercise: Given a tiny vocabulary {cat, dog, pet} with co-occurrence counts:

$$X_{\text{cat,pet}} = 50, \quad X_{\text{dog,pet}} = 45, \quad X_{\text{cat,dog}} = 10$$

and $x_{\max} = 100$, $\alpha = 0.75$.

Compute:

1. $f(X_{\text{cat,pet}})$ and $f(X_{\text{cat,dog}})$
2. If current vectors give $\mathbf{w}_{\text{cat}}^{\top}\tilde{\mathbf{w}}_{\text{pet}} + b_{\text{cat}} + \tilde{b}_{\text{pet}} = 3.5$ and $\log 50 \approx 3.91$, what is the loss contribution from this pair?
3. Which pair contributes most to training? Why?

Practice: GloVe Solutions

1. Weighting function:

$$f(50) = (50/100)^{0.75} = 0.5^{0.75} \approx 0.59$$

$$f(10) = (10/100)^{0.75} = 0.1^{0.75} \approx 0.18$$

Practice: GloVe Solutions

1. Weighting function:

$$f(50) = (50/100)^{0.75} = 0.5^{0.75} \approx 0.59$$

$$f(10) = (10/100)^{0.75} = 0.1^{0.75} \approx 0.18$$

2. Loss for (cat, pet):

$$f(50) \cdot (3.5 - 3.91)^2 = 0.59 \times 0.168 \approx 0.099$$

Practice: GloVe Solutions

1. Weighting function:

$$f(50) = (50/100)^{0.75} = 0.5^{0.75} \approx 0.59$$

$$f(10) = (10/100)^{0.75} = 0.1^{0.75} \approx 0.18$$

2. Loss for (cat, pet):

$$f(50) \cdot (3.5 - 3.91)^2 = 0.59 \times 0.168 \approx 0.099$$

3. The (cat, pet) pair contributes most because:

- Higher co-occurrence $\Rightarrow$ higher weight $f(X_{ij})$
- The weighting function ensures frequent, meaningful co-occurrences drive learning
- Rare pairs (cat, dog $= 10$) get downweighted ($f = 0.18$)

Practice: GloVe Edge Cases

1. What happens to the GloVe objective if you set $\alpha = 0$ in the weighting function? What about $\alpha = 1$? Which is worse and why?

Practice: GloVe Edge Cases

1. What happens to the GloVe objective if you set $\alpha = 0$ in the weighting function? What about $\alpha = 1$? Which is worse and why?

2. GloVe trains *two* vectors per word: $\mathbf{w}_i$ and $\tilde{\mathbf{w}}_i$. The final embedding is $\mathbf{w}_i + \tilde{\mathbf{w}}_i$. Why not just use $\mathbf{w}_i$? (Hint: think about the symmetry of co-occurrence.)

Practice: GloVe Edge Cases

1. What happens to the GloVe objective if you set $\alpha = 0$ in the weighting function? What about $\alpha = 1$? Which is worse and why?

2. GloVe trains *two* vectors per word: $\mathbf{w}_i$ and $\tilde{\mathbf{w}}_i$. The final embedding is $\mathbf{w}_i + \tilde{\mathbf{w}}_i$. Why not just use $\mathbf{w}_i$? (Hint: think about the symmetry of co-occurrence.)

3. You train GloVe on English Wikipedia vs. Twitter data. For each pair, predict which corpus gives higher cosine similarity:

- ("president", "executive") — Wikipedia or Twitter?
- ("lol", "lmao") — Wikipedia or Twitter?
- ("bank", "river") — Wikipedia or Twitter?

What does this tell you about the relationship between training data and embedding quality?

Discussion: Choosing Your Embedding Method

Group discussion (3 min): You're starting a new NLP project. Under what circumstances would you choose each method?

Discussion: Choosing Your Embedding Method

Group discussion (3 min): You're starting a new NLP project. Under what circumstances would you choose each method?

Scenarios to consider:

1. You have a small, domain-specific corpus (e.g., legal documents)
2. You have massive web-scale data and need fast training
3. You care about capturing global statistical properties
4. You want to update embeddings incrementally as new data arrives
5. You need interpretable dimensions

Discussion: Choosing Your Embedding Method

Group discussion (3 min): You're starting a new NLP project. Under what circumstances would you choose each method?

Scenarios to consider:

1. You have a small, domain-specific corpus (e.g., legal documents)
2. You have massive web-scale data and need fast training
3. You care about capturing global statistical properties
4. You want to update embeddings incrementally as new data arrives
5. You need interpretable dimensions

Follow-up: In practice, most people download pre-trained embeddings. When does training your own matter?

Neural Networks: The Basics

A feed-forward neural network computes:

$$\mathbf{h}^{(1)} = g\left(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}\right)$$

$$\mathbf{h}^{(2)} = g\left(\mathbf{W}^{(2)}\mathbf{h}^{(1)} + \mathbf{b}^{(2)}\right)$$

$$\hat{\mathbf{y}} = \text{softmax}\left(\mathbf{W}^{(3)}\mathbf{h}^{(2)} + \mathbf{b}^{(3)}\right)$$

Neural Networks: The Basics

A feed-forward neural network computes:

$$\mathbf{h}^{(1)} = g\big(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}\big)$$

$$\mathbf{h}^{(2)} = g\big(\mathbf{W}^{(2)}\mathbf{h}^{(1)} + \mathbf{b}^{(2)}\big)$$

$$\hat{\mathbf{y}} = \text{softmax}\big(\mathbf{W}^{(3)}\mathbf{h}^{(2)} + \mathbf{b}^{(3)}\big)$$

Key components:

- **Linear transformation:** $\mathbf{W}\mathbf{x} + \mathbf{b}$ (affine map)
- **Activation function** $g(\cdot)$: introduces non-linearity
- **Output layer:** softmax for classification, linear for regression

Activation Functions

Sigmoid:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Output in $(0, 1)$. Historically popular, but suffers from *vanishing gradients*.

Tanh:

$$\tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$

Output in $(-1, 1)$. Zero-centered, but still saturates.

ReLU:

$$\mathrm{ReLU}(x) = \max(0, x)$$

Simple, fast, no saturation for $x > 0$. Most commonly used today.

Activation Functions

Sigmoid:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Output in $(0, 1)$. Historically popular, but suffers from *vanishing gradients*.

Tanh:

$$\tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$

Output in $(-1, 1)$. Zero-centered, but still saturates.

ReLU:

$$\mathrm{ReLU}(x) = \max(0, x)$$

Simple, fast, no saturation for $x > 0$. Most commonly used today.

Why non-linearity?
Without it, stacking linear layers just gives another linear function. Non-linearity lets networks learn **complex decision boundaries**.

Backpropagation

Goal: Compute $\frac{\partial \mathcal{L}}{\partial \theta}$ for every parameter θ to do gradient descent.

Backpropagation

Goal: Compute $\frac{\partial \mathcal{L}}{\partial \theta}$ for every parameter θ to do gradient descent.

Chain rule applied layer by layer (backward):

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}} = \frac{\partial \mathcal{L}}{\partial \hat{\mathbf{y}}} \cdot \frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{h}^{(2)}} \cdot \frac{\partial \mathbf{h}^{(2)}}{\partial \mathbf{h}^{(1)}} \cdot \frac{\partial \mathbf{h}^{(1)}}{\partial \mathbf{W}^{(1)}}$$

Backpropagation

Goal: Compute $\frac{\partial \mathcal{L}}{\partial \theta}$ for every parameter θ to do gradient descent.

Chain rule applied layer by layer (backward):

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}} = \frac{\partial \mathcal{L}}{\partial \hat{\mathbf{y}}} \cdot \frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{h}^{(2)}} \cdot \frac{\partial \mathbf{h}^{(2)}}{\partial \mathbf{h}^{(1)}} \cdot \frac{\partial \mathbf{h}^{(1)}}{\partial \mathbf{W}^{(1)}}$$

The algorithm:

1. **Forward pass:** Compute all activations, store intermediate values
2. **Compute loss:** $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$ (e.g., cross-entropy)
3. **Backward pass:** Propagate gradients from output to input
4. **Update:** $\theta \leftarrow \theta - \eta \frac{\partial \mathcal{L}}{\partial \theta}$

Backpropagation

Goal: Compute $\frac{\partial \mathcal{L}}{\partial \theta}$ for every parameter θ to do gradient descent.

Chain rule applied layer by layer (backward):

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}} = \frac{\partial \mathcal{L}}{\partial \hat{\mathbf{y}}} \cdot \frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{h}^{(2)}} \cdot \frac{\partial \mathbf{h}^{(2)}}{\partial \mathbf{h}^{(1)}} \cdot \frac{\partial \mathbf{h}^{(1)}}{\partial \mathbf{W}^{(1)}}$$

The algorithm:

1. **Forward pass:** Compute all activations, store intermediate values
2. **Compute loss:** $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$ (e.g., cross-entropy)
3. **Backward pass:** Propagate gradients from output to input
4. **Update:** $\theta \leftarrow \theta - \eta \frac{\partial \mathcal{L}}{\partial \theta}$

Every parameter in the network — including **embeddings** — gets updated this way.

Worked Example: Full Forward & Backward Pass

Setup: A tiny network for binary sentiment classification.

- Input: $\mathbf{x} = \begin{pmatrix} 2 \\ 1 \end{pmatrix}$, True label: $y = 1$ (positive)
- Hidden layer: $\mathbf{W}^{(1)} = \begin{pmatrix} 0.5 & -0.3 \\ 0.2 & 0.4 \end{pmatrix}$, $\mathbf{b}^{(1)} = \begin{pmatrix} 0.1 \\ -0.1 \end{pmatrix}$
- Output layer: $\mathbf{w}^{(2)} = \begin{pmatrix} 0.6 \\ -0.2 \end{pmatrix}$, $b^{(2)} = 0.1$
- Output activation: sigmoid, Loss: binary cross-entropy

Architecture:

$$\mathbf{x} \xrightarrow{\mathbf{W}^{(1)}, \mathbf{b}^{(1)}} \mathbf{z}^{(1)} \xrightarrow{g(\cdot)} \mathbf{h}^{(1)} \xrightarrow{\mathbf{w}^{(2)}, b^{(2)}} z^{(2)} \xrightarrow{\sigma} \hat{y}$$

We'll do this three times, swapping $g(\cdot)$ between **ReLU**, **sigmoid**, and **tanh**.

Forward Pass (Shared Across All Activations)

Step 1: Pre-activation

$$\mathbf{z}^{(1)} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)} = \begin{pmatrix} 0.5 & -0.3 \\ 0.2 & 0.4 \end{pmatrix} \begin{pmatrix} 2 \\ 1 \end{pmatrix} + \begin{pmatrix} 0.1 \\ -0.1 \end{pmatrix} = \begin{pmatrix} 0.7 + 0.1 \\ 0.8 - 0.1 \end{pmatrix} = \begin{pmatrix} 0.8 \\ 0.7 \end{pmatrix}$$

Forward Pass (Shared Across All Activations)

Step 1: Pre-activation

$$\mathbf{z}^{(1)} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)} = \begin{pmatrix} 0.5 & -0.3 \\ 0.2 & 0.4 \end{pmatrix} \begin{pmatrix} 2 \\ 1 \end{pmatrix} + \begin{pmatrix} 0.1 \\ -0.1 \end{pmatrix} = \begin{pmatrix} 0.7 + 0.1 \\ 0.8 - 0.1 \end{pmatrix} = \begin{pmatrix} 0.8 \\ 0.7 \end{pmatrix}$$

Step 2: Hidden activation $\mathbf{h}^{(1)} = g(\mathbf{z}^{(1)})$ — depends on choice of g!

$g(\cdot)$	$h_1 = g(0.8)$	$h_2 = g(0.7)$
ReLU	$\max(0, 0.8) = 0.8$	$\max(0, 0.7) = 0.7$
Sigmoid	$\sigma(0.8) \approx 0.690$	$\sigma(0.7) \approx 0.668$
Tanh	$\tanh(0.8) \approx 0.664$	$\tanh(0.7) \approx 0.604$

Forward Pass (Shared Across All Activations)

Step 1: Pre-activation

$$\mathbf{z}^{(1)} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)} = \begin{pmatrix} 0.5 & -0.3 \\ 0.2 & 0.4 \end{pmatrix} \begin{pmatrix} 2 \\ 1 \end{pmatrix} + \begin{pmatrix} 0.1 \\ -0.1 \end{pmatrix} = \begin{pmatrix} 0.7 + 0.1 \\ 0.8 - 0.1 \end{pmatrix} = \begin{pmatrix} 0.8 \\ 0.7 \end{pmatrix}$$

Step 2: Hidden activation $\mathbf{h}^{(1)} = g(\mathbf{z}^{(1)})$ — depends on choice of g!

$g(\cdot)$	$h_1 = g(0.8)$	$h_2 = g(0.7)$
ReLU	$\max(0, 0.8) = 0.8$	$\max(0, 0.7) = 0.7$
Sigmoid	$\sigma(0.8) \approx 0.690$	$\sigma(0.7) \approx 0.668$
Tanh	$\tanh(0.8) \approx 0.664$	$\tanh(0.7) \approx 0.604$

Notice: ReLU passes values through unchanged (when positive). Sigmoid and tanh both **squash** the values, with sigmoid shifting everything into $(0, 1)$.

Forward Pass: Output and Loss

Step 3: Output (using ReLU hidden values for now):

$$z^{(2)} = \mathbf{w}^{(2)\top}\mathbf{h}^{(1)} + b^{(2)} = 0.6(0.8) + (-0.2)(0.7) + 0.1 = 0.44$$

$$\hat{y} = \sigma(0.44) \approx 0.608$$

Forward Pass: Output and Loss

Step 3: Output (using ReLU hidden values for now):

$$z^{(2)} = \mathbf{w}^{(2)\top}\mathbf{h}^{(1)} + b^{(2)} = 0.6(0.8) + (-0.2)(0.7) + 0.1 = 0.44$$

$$\hat{y} = \sigma(0.44) \approx 0.608$$

Step 4: Loss ($y = 1$):

$$\mathcal{L} = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})] = -\log(0.608) \approx 0.497$$

Forward Pass: Output and Loss

Step 3: Output (using ReLU hidden values for now):

$$z^{(2)} = \mathbf{w}^{(2)\top}\mathbf{h}^{(1)} + b^{(2)} = 0.6(0.8) + (-0.2)(0.7) + 0.1 = 0.44$$

$$\hat{y} = \sigma(0.44) \approx 0.608$$

Step 4: Loss ($y = 1$):

$$\mathcal{L} = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})] = -\log(0.608) \approx 0.497$$

We predicted $P(y = 1) = 0.608$ for a true positive. Not terrible, but not confident — the gradient should push the network to be more confident.

Backward Pass: Output Layer (Same for All)

Step 5: Gradient of loss w.r.t. output:

$$\frac{\partial \mathcal{L}}{\partial z^{(2)}} = \hat{y} - y = 0.608 - 1 = -0.392$$

Backward Pass: Output Layer (Same for All)

Step 5: Gradient of loss w.r.t. output:

$$\frac{\partial \mathcal{L}}{\partial z^{(2)}} = \hat{y} - y = 0.608 - 1 = -0.392$$

Step 6: Gradients for output weights (using ReLU $\mathbf{h}^{(1)} = (0.8, 0.7)$):

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}^{(2)}} = \frac{\partial \mathcal{L}}{\partial z^{(2)}} \cdot \mathbf{h}^{(1)} = -0.392 \begin{pmatrix} 0.8 \\ 0.7 \end{pmatrix} = \begin{pmatrix} -0.314 \\ -0.274 \end{pmatrix}$$

$$\frac{\partial \mathcal{L}}{\partial b^{(2)}} = -0.392$$

Backward Pass: Output Layer (Same for All)

Step 5: Gradient of loss w.r.t. output:

$$\frac{\partial \mathcal{L}}{\partial z^{(2)}} = \hat{y} - y = 0.608 - 1 = -0.392$$

Step 6: Gradients for output weights (using ReLU $\mathbf{h}^{(1)} = (0.8, 0.7)$):

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}^{(2)}} = \frac{\partial \mathcal{L}}{\partial z^{(2)}} \cdot \mathbf{h}^{(1)} = -0.392 \begin{pmatrix} 0.8 \\ 0.7 \end{pmatrix} = \begin{pmatrix} -0.314 \\ -0.274 \end{pmatrix}$$

$$\frac{\partial \mathcal{L}}{\partial b^{(2)}} = -0.392$$

Step 7: Error signal passed back to hidden layer:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(1)}} = \frac{\partial \mathcal{L}}{\partial z^{(2)}} \cdot \mathbf{w}^{(2)} = -0.392 \begin{pmatrix} 0.6 \\ -0.2 \end{pmatrix} = \begin{pmatrix} -0.235 \\ 0.078 \end{pmatrix}$$

Backward Pass: Through the Activation (This Is Where It Differs!)

Step 8: To get $\frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(1)}}$, we multiply by $g'(\mathbf{z}^{(1)})$ element-wise:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(1)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(1)}} \odot g'(\mathbf{z}^{(1)})$$

Backward Pass: Through the Activation (This Is Where It Differs!)

Step 8: To get $\frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(1)}}$, we multiply by $g'(\mathbf{z}^{(1)})$ element-wise:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(1)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(1)}} \odot g'(\mathbf{z}^{(1)})$$

Recall $\mathbf{z}^{(1)} = (0.8, 0.7)$ and $\frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(1)}} = (-0.235,\ 0.078)$.

Activation derivatives:

g	$g'(z)$	$g'(0.8)$	$g'(0.7)$
ReLU	$\begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}$	1.0	1.0
Sigmoid	$\sigma(z)(1-\sigma(z))$	0.214	0.222
Tanh	$1 - \tanh^2(z)$	0.559	0.635

Backward Pass: The Gradient Comparison

Gradient through the activation: $\frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(1)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(1)}} \odot g'(\mathbf{z}^{(1)})$

g	$\frac{\partial \mathcal{L}}{\partial z_1^{(1)}}$	$\frac{\partial \mathcal{L}}{\partial z_2^{(1)}}$
ReLU	$-0.235 \times 1.0 = \mathbf{-0.235}$	$0.078 \times 1.0 = \mathbf{0.078}$
Sigmoid	$-0.235 \times 0.214 = \mathbf{-0.050}$	$0.078 \times 0.222 = \mathbf{0.017}$
Tanh	$-0.235 \times 0.559 = \mathbf{-0.131}$	$0.078 \times 0.635 = \mathbf{0.050}$

Backward Pass: The Gradient Comparison

Gradient through the activation: $\frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(1)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(1)}} \odot g'(\mathbf{z}^{(1)})$

g	$\frac{\partial \mathcal{L}}{\partial z_1^{(1)}}$	$\frac{\partial \mathcal{L}}{\partial z_2^{(1)}}$
ReLU	$-0.235 \times 1.0 = \mathbf{-0.235}$	$0.078 \times 1.0 = \mathbf{0.078}$
Sigmoid	$-0.235 \times 0.214 = \mathbf{-0.050}$	$0.078 \times 0.222 = \mathbf{0.017}$
Tanh	$-0.235 \times 0.559 = \mathbf{-0.131}$	$0.078 \times 0.635 = \mathbf{0.050}$

Key observation:

- ReLU: **full gradient** passes through (no shrinkage)
- Sigmoid: gradient **shrinks by** $\sim$**4–5**$\times$ (max derivative is 0.25!)
- Tanh: gradient **shrinks by** $\sim$**2**$\times$ (better than sigmoid, worse than ReLU)

Backward Pass: The Gradient Comparison

Gradient through the activation: $\frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(1)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(1)}} \odot g'(\mathbf{z}^{(1)})$

g	$\frac{\partial \mathcal{L}}{\partial z_1^{(1)}}$	$\frac{\partial \mathcal{L}}{\partial z_2^{(1)}}$
ReLU	$-0.235 \times 1.0 = \mathbf{-0.235}$	$0.078 \times 1.0 = \mathbf{0.078}$
Sigmoid	$-0.235 \times 0.214 = \mathbf{-0.050}$	$0.078 \times 0.222 = \mathbf{0.017}$
Tanh	$-0.235 \times 0.559 = \mathbf{-0.131}$	$0.078 \times 0.635 = \mathbf{0.050}$

Key observation:

- ReLU: **full gradient** passes through (no shrinkage)
- Sigmoid: gradient **shrinks by** $\sim$**4–5**$\times$ (max derivative is 0.25!)
- Tanh: gradient **shrinks by** $\sim$**2**$\times$ (better than sigmoid, worse than ReLU)

This is the vanishing gradient problem in action — and it gets exponentially worse with more layers!

Weight Updates: The Final Numbers

Step 9: Gradient for hidden weights: $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(1)}} \cdot \mathbf{x}^\top$

With ReLU:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}} = \begin{pmatrix} -0.235 \\ 0.078 \end{pmatrix} \begin{pmatrix} 2 & 1 \end{pmatrix} = \begin{pmatrix} -0.470 & -0.235 \\ 0.157 & 0.078 \end{pmatrix}$$

Weight Updates: The Final Numbers

Step 9: Gradient for hidden weights: $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(1)}} \cdot \mathbf{x}^\top$

With ReLU:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}} = \begin{pmatrix} -0.235 \\ 0.078 \end{pmatrix} \begin{pmatrix} 2 & 1 \end{pmatrix} = \begin{pmatrix} -0.470 & -0.235 \\ 0.157 & 0.078 \end{pmatrix}$$

With sigmoid:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}} = \begin{pmatrix} -0.050 \\ 0.017 \end{pmatrix} \begin{pmatrix} 2 & 1 \end{pmatrix} = \begin{pmatrix} -0.101 & -0.050 \\ 0.035 & 0.017 \end{pmatrix}$$

Weight Updates: The Final Numbers

Step 9: Gradient for hidden weights: $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(1)}} \cdot \mathbf{x}^\top$

With ReLU:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}} = \begin{pmatrix} -0.235 \\ 0.078 \end{pmatrix} \begin{pmatrix} 2 & 1 \end{pmatrix} = \begin{pmatrix} -0.470 & -0.235 \\ 0.157 & 0.078 \end{pmatrix}$$

With sigmoid:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}} = \begin{pmatrix} -0.050 \\ 0.017 \end{pmatrix} \begin{pmatrix} 2 & 1 \end{pmatrix} = \begin{pmatrix} -0.101 & -0.050 \\ 0.035 & 0.017 \end{pmatrix}$$

With tanh:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}} = \begin{pmatrix} -0.131 \\ 0.050 \end{pmatrix} \begin{pmatrix} 2 & 1 \end{pmatrix} = \begin{pmatrix} -0.263 & -0.131 \\ 0.099 & 0.050 \end{pmatrix}$$

Weight Updates: SGD with $\eta = 0.5$

Update rule: $\mathbf{W}^{(1)}_{\text{new}} = \mathbf{W}^{(1)} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}}$

ReLU (biggest updates):

$$\mathbf{W}^{(1)}_{\text{new}} = \begin{pmatrix} 0.5 & -0.3 \\ 0.2 & 0.4 \end{pmatrix} - 0.5 \begin{pmatrix} -0.470 & -0.235 \\ 0.157 & 0.078 \end{pmatrix} = \begin{pmatrix} 0.735 & -0.182 \\ 0.122 & 0.361 \end{pmatrix}$$

Weight Updates: SGD with $\eta = 0.5$

Update rule: $\mathbf{W}^{(1)}_{\text{new}} = \mathbf{W}^{(1)} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}}$

ReLU (biggest updates):

$$\mathbf{W}^{(1)}_{\text{new}} = \begin{pmatrix} 0.5 & -0.3 \\ 0.2 & 0.4 \end{pmatrix} - 0.5 \begin{pmatrix} -0.470 & -0.235 \\ 0.157 & 0.078 \end{pmatrix} = \begin{pmatrix} 0.735 & -0.182 \\ 0.122 & 0.361 \end{pmatrix}$$

Sigmoid (smallest updates):

$$\mathbf{W}^{(1)}_{\text{new}} = \begin{pmatrix} 0.5 & -0.3 \\ 0.2 & 0.4 \end{pmatrix} - 0.5 \begin{pmatrix} -0.101 & -0.050 \\ 0.035 & 0.017 \end{pmatrix} = \begin{pmatrix} 0.550 & -0.275 \\ 0.183 & 0.391 \end{pmatrix}$$

Weight Updates: SGD with $\eta = 0.5$

Update rule: $\mathbf{W}^{(1)}_{\text{new}} = \mathbf{W}^{(1)} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}}$

ReLU (biggest updates):

$$\mathbf{W}^{(1)}_{\text{new}} = \begin{pmatrix} 0.5 & -0.3 \\ 0.2 & 0.4 \end{pmatrix} - 0.5 \begin{pmatrix} -0.470 & -0.235 \\ 0.157 & 0.078 \end{pmatrix} = \begin{pmatrix} 0.735 & -0.182 \\ 0.122 & 0.361 \end{pmatrix}$$

Sigmoid (smallest updates):

$$\mathbf{W}^{(1)}_{\text{new}} = \begin{pmatrix} 0.5 & -0.3 \\ 0.2 & 0.4 \end{pmatrix} - 0.5 \begin{pmatrix} -0.101 & -0.050 \\ 0.035 & 0.017 \end{pmatrix} = \begin{pmatrix} 0.550 & -0.275 \\ 0.183 & 0.391 \end{pmatrix}$$

ReLU's W_{11}: $0.5 \to 0.735$ $\quad (+47\%)$
Sigmoid's W_{11}: $0.5 \to 0.550$ $\quad (+10\%)$

Same network, same error, same learning rate — sigmoid learns $\sim 5\times$ slower in the hidden layer.

What If $z^{(1)}$ Had Been Negative?

Our example had $\mathbf{z}^{(1)} = (0.8, 0.7)$, both positive. What if $z_1^{(1)} = -2.0$?

What If $z^{(1)}$ Had Been Negative?

Our example had $\mathbf{z}^{(1)} = (0.8, 0.7)$, both positive. What if $z_1^{(1)} = -2.0$?

g	$g'(-2.0)$	Effect on gradient
ReLU	0	**Gradient is killed completely ("dead neuron")**
Sigmoid	$\sigma(-2)(1-\sigma(-2)) \approx 0.105$	Small but nonzero
Tanh	$1 - \tanh^2(-2) \approx 0.071$	Small but nonzero

What If $z^{(1)}$ Had Been Negative?

Our example had $\mathbf{z}^{(1)} = (0.8, 0.7)$, both positive. What if $z_1^{(1)} = -2.0$?

g	$g'(-2.0)$	Effect on gradient
ReLU	0	**Gradient is killed completely ("dead neuron")**
Sigmoid	$\sigma(-2)(1-\sigma(-2)) \approx 0.105$	Small but nonzero
Tanh	$1 - \tanh^2(-2) \approx 0.071$	Small but nonzero

The ReLU tradeoff:

- When $z > 0$: perfect gradient flow, no shrinkage
- When $z \leq 0$: **zero gradient** — the neuron is "dead" and stops learning
- If many neurons die $\Rightarrow$ "dying ReLU" problem
- Fix: Leaky ReLU ($g(z) = \max(0.01z, z)$) or other variants

What If $z^{(1)}$ Had Been Negative?

Our example had $\mathbf{z}^{(1)} = (0.8, 0.7)$, both positive. What if $z_1^{(1)} = -2.0$?

g	$g'(-2.0)$	Effect on gradient
ReLU	0	**Gradient is killed completely ("dead neuron")**
Sigmoid	$\sigma(-2)(1-\sigma(-2)) \approx 0.105$	Small but nonzero
Tanh	$1 - \tanh^2(-2) \approx 0.071$	Small but nonzero

The ReLU tradeoff:

- When $z > 0$: perfect gradient flow, no shrinkage
- When $z \leq 0$: **zero gradient** — the neuron is "dead" and stops learning
- If many neurons die $\Rightarrow$ "dying ReLU" problem
- Fix: Leaky ReLU ($g(z) = \max(0.01z, z)$) or other variants

No activation function is perfect — each makes a different tradeoff between gradient magnitude and information preservation.

Embedding Layers: The Bridge

How do word vectors enter a neural network?

Embedding Layers: The Bridge

How do word vectors enter a neural network?

1. Start with a word index: $w = 42$ ("cat")
2. **One-hot encoding:** $\mathbf{x} \in \{0, 1\}^{|V|}$, with $x_{42} = 1$
3. **Embedding lookup:** $\mathbf{e} = \mathbf{E}^\top \mathbf{x} = \mathbf{E}[42]$

Embedding Layers: The Bridge

How do word vectors enter a neural network?

1. Start with a word index: $w = 42$ ("cat")
2. **One-hot encoding:** $\mathbf{x} \in \{0, 1\}^{|V|}$, with $x_{42} = 1$
3. **Embedding lookup:** $\mathbf{e} = \mathbf{E}^\top \mathbf{x} = \mathbf{E}[42]$

The embedding matrix $\mathbf{E} \in \mathbb{R}^{|V| \times d}$ is just a **lookup table**:

- Row i = the d-dimensional vector for word i
- Multiplying by a one-hot vector selects that row
- $\mathbf{E}$ is a **learnable parameter** — updated by backprop!

Embedding Layers: The Bridge

How do word vectors enter a neural network?

1. Start with a word index: $w = 42$ ("cat")
2. **One-hot encoding:** $\mathbf{x} \in \{0, 1\}^{|V|}$, with $x_{42} = 1$
3. **Embedding lookup:** $\mathbf{e} = \mathbf{E}^\top \mathbf{x} = \mathbf{E}[42]$

The embedding matrix $\mathbf{E} \in \mathbb{R}^{|V| \times d}$ is just a **lookup table**:

- Row i = the d-dimensional vector for word i
- Multiplying by a one-hot vector selects that row
- $\mathbf{E}$ is a **learnable parameter** — updated by backprop!

Key Insight: Word2Vec *is* a neural network! Skip-gram = one-hot $\rightarrow$ embedding $\rightarrow$ dot product $\rightarrow$ sigmoid. The "embedding" is just the hidden layer weights.

Word2Vec IS a Neural Network



embedding layer

prediction

Word2Vec IS a Neural Network



- $\mathbf{W} \in \mathbb{R}^{d \times |V|}$ — this **is** the embedding matrix
- $\mathbf{W}' \in \mathbb{R}^{|V| \times d}$ — context embedding matrix
- No hidden non-linearity — the "depth" comes from the learning objective
- The embeddings are the **byproduct** of training this shallow network

Pre-trained vs. Learned-from-Scratch Embeddings

Pre-trained (GloVe, Word2Vec):

- Trained on billions of words
- Capture general semantic knowledge
- Initialize **E** then optionally **fine-tune**
- Great when your labeled data is small

Learned from scratch:

- Random initialization of **E**
- Learns task-specific representations
- Needs lots of labeled data
- May miss general knowledge

Pre-trained vs. Learned-from-Scratch Embeddings

Pre-trained (GloVe, Word2Vec):

- Trained on billions of words
- Capture general semantic knowledge
- Initialize **E** then optionally **fine-tune**
- Great when your labeled data is small

Learned from scratch:

- Random initialization of **E**
- Learns task-specific representations
- Needs lots of labeled data
- May miss general knowledge

Common strategy: Initialize with pre-trained vectors, then fine-tune on your task. Best of both worlds: general knowledge + task-specific adaptation.

Backprop Through the Embedding Layer

When we fine-tune, gradients flow **all the way back to E**:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{E}[w]} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(1)}} \cdot \frac{\partial \mathbf{h}^{(1)}}{\partial \mathbf{e}_w} \cdot \frac{\partial \mathbf{e}_w}{\partial \mathbf{E}[w]}$$

Backprop Through the Embedding Layer

When we fine-tune, gradients flow **all the way back to E**:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{E}[w]} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(1)}} \cdot \frac{\partial \mathbf{h}^{(1)}}{\partial \mathbf{e}_w} \cdot \frac{\partial \mathbf{e}_w}{\partial \mathbf{E}[w]}$$

What happens during fine-tuning:

- Only the rows for words *seen in training* get updated
- Words relevant to the task shift in embedding space
- "bank" in a finance classifier drifts toward financial context
- Rare words barely move (few gradient updates)

Backprop Through the Embedding Layer

When we fine-tune, gradients flow **all the way back to E**:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{E}[w]} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(1)}} \cdot \frac{\partial \mathbf{h}^{(1)}}{\partial \mathbf{e}_w} \cdot \frac{\partial \mathbf{e}_w}{\partial \mathbf{E}[w]}$$

What happens during fine-tuning:

- Only the rows for words *seen in training* get updated
- Words relevant to the task shift in embedding space
- "bank" in a finance classifier drifts toward financial context
- Rare words barely move (few gradient updates)

Frozen vs. fine-tuned: You can also **freeze E** (no gradient updates) to prevent overfitting on small datasets. This uses pre-trained vectors as fixed features.

Practice: Forward Pass Through an Embedding Classifier

Exercise: A sentiment classifier takes a single word as input. Given:

- Vocabulary: {"good" = 0, "bad" = 1, "great" = 2}
- Embedding matrix $\mathbf{E} = \begin{pmatrix} 0.5 & 0.3 \\ -0.4 & 0.2 \\ 0.6 & 0.5 \end{pmatrix}$ (3×2)
- Hidden weights $\mathbf{W}^{(1)} = \begin{pmatrix} 1.0 & -0.5 \\ 0.3 & 0.8 \end{pmatrix}$, bias $\mathbf{b}^{(1)} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$
- ReLU activation

For input "great" (index 2):

1. What is the embedding vector $\mathbf{e}$?
2. Compute $\mathbf{z}^{(1)} = \mathbf{W}^{(1)}\mathbf{e} + \mathbf{b}^{(1)}$
3. Apply ReLU to get $\mathbf{h}^{(1)}$

Practice: Forward Pass Solutions

1. **Embedding lookup:** $\mathbf{e} = \mathbf{E}[2] = (0.6,\ 0.5)$

Practice: Forward Pass Solutions

1. **Embedding lookup:** $\mathbf{e} = \mathbf{E}[2] = (0.6,\ 0.5)$

2. **Linear transformation:**

$$\mathbf{z}^{(1)} = \begin{pmatrix} 1.0 & -0.5 \\ 0.3 & 0.8 \end{pmatrix} \begin{pmatrix} 0.6 \\ 0.5 \end{pmatrix} + \begin{pmatrix} 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 0.6 - 0.25 \\ 0.18 + 0.4 \end{pmatrix} = \begin{pmatrix} 0.35 \\ 0.58 \end{pmatrix}$$

Practice: Forward Pass Solutions

1. **Embedding lookup:** $\mathbf{e} = \mathbf{E}[2] = (0.6,\ 0.5)$

2. **Linear transformation:**

$$\mathbf{z}^{(1)} = \begin{pmatrix} 1.0 & -0.5 \\ 0.3 & 0.8 \end{pmatrix} \begin{pmatrix} 0.6 \\ 0.5 \end{pmatrix} + \begin{pmatrix} 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 0.6 - 0.25 \\ 0.18 + 0.4 \end{pmatrix} = \begin{pmatrix} 0.35 \\ 0.58 \end{pmatrix}$$

3. **ReLU activation:**

$$\mathbf{h}^{(1)} = \text{ReLU} \begin{pmatrix} 0.35 \\ 0.58 \end{pmatrix} = \begin{pmatrix} 0.35 \\ 0.58 \end{pmatrix}$$

(Both values are positive, so ReLU leaves them unchanged.)

Practice: Forward Pass Solutions

1. **Embedding lookup:** $\mathbf{e} = \mathbf{E}[2] = (0.6,\ 0.5)$

2. **Linear transformation:**

$$\mathbf{z}^{(1)} = \begin{pmatrix} 1.0 & -0.5 \\ 0.3 & 0.8 \end{pmatrix} \begin{pmatrix} 0.6 \\ 0.5 \end{pmatrix} + \begin{pmatrix} 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 0.6 - 0.25 \\ 0.18 + 0.4 \end{pmatrix} = \begin{pmatrix} 0.35 \\ 0.58 \end{pmatrix}$$

3. **ReLU activation:**

$$\mathbf{h}^{(1)} = \text{ReLU} \begin{pmatrix} 0.35 \\ 0.58 \end{pmatrix} = \begin{pmatrix} 0.35 \\ 0.58 \end{pmatrix}$$

(Both values are positive, so ReLU leaves them unchanged.)

Next step (not shown): this feeds into an output layer $\rightarrow$ softmax $\rightarrow$ sentiment prediction.

Practice: Embedding Fine-tuning Predictions

Setup: You initialize a sentiment classifier with pre-trained GloVe embeddings and fine-tune on movie reviews.

1. After fine-tuning, predict the **direction** each word's vector moves:

- "brilliant" (appears 500 times, always in positive reviews)
- "terrible" (appears 400 times, always in negative reviews)
- "film" (appears 2000 times, equally in positive and negative)
- "incandescent" (appears 2 times, both in positive reviews)

Practice: Embedding Fine-tuning Predictions

Setup: You initialize a sentiment classifier with pre-trained GloVe embeddings and fine-tune on movie reviews.

1. After fine-tuning, predict the **direction** each word's vector moves:

- "brilliant" (appears 500 times, always in positive reviews)
- "terrible" (appears 400 times, always in negative reviews)
- "film" (appears 2000 times, equally in positive and negative)
- "incandescent" (appears 2 times, both in positive reviews)

2. You freeze the embedding layer instead. What changes about the model's ability to learn? When would freezing outperform fine-tuning?

Practice: Embedding Fine-tuning Predictions

Setup: You initialize a sentiment classifier with pre-trained GloVe embeddings and fine-tune on movie reviews.

1. After fine-tuning, predict the **direction** each word's vector moves:

- "brilliant" (appears 500 times, always in positive reviews)
- "terrible" (appears 400 times, always in negative reviews)
- "film" (appears 2000 times, equally in positive and negative)
- "incandescent" (appears 2 times, both in positive reviews)

2. You freeze the embedding layer instead. What changes about the model's ability to learn? When would freezing outperform fine-tuning?

3. A word appears in your test set that was never in the training data. What happens with fine-tuned embeddings? With frozen embeddings? Which handles this better?

Discussion: Embedding Geometry

Discuss with your neighbor (3 min): Why do embeddings generalize? What does the geometry of embedding space tell us?

Discussion: Embedding Geometry

Discuss with your neighbor (3 min): Why do embeddings generalize? What does the geometry of embedding space tell us?

Consider:

- Why do similar words cluster together in embedding space?
- What does it mean for an embedding to be "good" for a downstream task?
- If you train embeddings on Twitter vs. medical text, how would "patient" differ?
- What are the limitations of static (non-contextual) embeddings?
  (Hint: polysemy — "bank" gets *one* vector for all its senses)
- How does this motivate contextual embeddings (ELMo, BERT)?

Activity: Tensorflow Embedding Projector

Let's explore real embeddings interactively.

`https://projector.tensorflow.org`

Activity: Tensorflow Embedding Projector

Let's explore real embeddings interactively.

`https://projector.tensorflow.org`

Things to try:

- Search for a word and look at its nearest neighbors
- Try the "custom projection" to see analogies in 2D/3D
- Switch between PCA and t-SNE — how does the visualization change?
- Look for clusters: do animals group together? Countries? Verbs?

Activity: Tensorflow Embedding Projector

Let's explore real embeddings interactively.

`https://projector.tensorflow.org`

Things to try:

- Search for a word and look at its nearest neighbors
- Try the "custom projection" to see analogies in 2D/3D
- Switch between PCA and t-SNE — how does the visualization change?
- Look for clusters: do animals group together? Countries? Verbs?

Challenges for your group:

1. Find a word whose nearest neighbors surprise you
2. Find an analogy that works and one that breaks
3. Find evidence of bias in the embeddings

Embedding Projector Debrief

What did you find?

Embedding Projector Debrief

What did you find?

Key takeaways:

- Embedding space has **meaningful geometric structure** — clusters, directions, subspaces
- **PCA** shows global structure (what are the biggest axes of variation?)
- **t-SNE** shows local structure (which words cluster together?)
- Biases in training data $\Rightarrow$ biases in embeddings (gender, race, culture)
- Static embeddings collapse all senses of a word into one point

Embedding Projector Debrief

What did you find?

Key takeaways:

- Embedding space has **meaningful geometric structure** — clusters, directions, subspaces
- **PCA** shows global structure (what are the biggest axes of variation?)
- **t-SNE** shows local structure (which words cluster together?)
- Biases in training data $\Rightarrow$ biases in embeddings (gender, race, culture)
- Static embeddings collapse all senses of a word into one point

These limitations motivate **contextual embeddings** — but that's for a future recitation.

Discussion: Looking Forward

Open discussion: How do these foundational ideas lead to modern NLP?

Discussion: Looking Forward

Open discussion: How do these foundational ideas lead to modern NLP?

Questions to explore:

1. Static embeddings give one vector per word. What's the problem with polysemy, and how might you solve it?
2. If Word2Vec is a shallow neural net, what happens if we go deeper?
3. What would it mean to learn embeddings that change based on context? (Preview: ELMo, BERT, Transformers)
4. Are there tasks where static embeddings are *still* preferred?

Discussion: Looking Forward

Open discussion: How do these foundational ideas lead to modern NLP?

Questions to explore:

1. Static embeddings give one vector per word. What's the problem with polysemy, and how might you solve it?
2. If Word2Vec is a shallow neural net, what happens if we go deeper?
3. What would it mean to learn embeddings that change based on context? (Preview: ELMo, BERT, Transformers)
4. Are there tasks where static embeddings are *still* preferred?

Key takeaway: Every modern language model — from BERT to GPT — builds on the distributional hypothesis. The methods change, but the core insight remains: **meaning is captured by context**.

Key Takeaways

Distributional Semantics:

- Co-occurrence matrices + PMI weighting + SVD = word vectors
- Captures meaning from usage patterns, no supervision needed

Key Takeaways

Distributional Semantics:

- Co-occurrence matrices + PMI weighting + SVD = word vectors
- Captures meaning from usage patterns, no supervision needed

Word2Vec & GloVe:

- Prediction-based methods that are implicitly factorizing PMI matrices
- Learn dense vectors with linear analogy structure

Key Takeaways

Distributional Semantics:

- Co-occurrence matrices + PMI weighting + SVD = word vectors
- Captures meaning from usage patterns, no supervision needed

Word2Vec & GloVe:

- Prediction-based methods that are implicitly factorizing PMI matrices
- Learn dense vectors with linear analogy structure

Neural Networks & Embeddings:

- Embedding layer = learnable lookup table, updated by backprop
- Word2Vec is itself a shallow neural network
- Pre-train on large data, fine-tune on your task

Key Takeaways

Distributional Semantics:

- Co-occurrence matrices + PMI weighting + SVD = word vectors
- Captures meaning from usage patterns, no supervision needed

Word2Vec & GloVe:

- Prediction-based methods that are implicitly factorizing PMI matrices
- Learn dense vectors with linear analogy structure

Neural Networks & Embeddings:

- Embedding layer = learnable lookup table, updated by backprop
- Word2Vec is itself a shallow neural network
- Pre-train on large data, fine-tune on your task

All are linear models of meaning — next up: what if we go deeper?

Questions?

Email: timothyobiso@brandeis.edu

Office Hours:
Tuesday 1:00-2:00 pm
Wednesday 2:00-3:00 pm